

current and exclusive information rather than standard responses. However, all component functions must be understood before users can learn to create a unified system.

## System architecture and RAG overview

The process of connecting an LLM to an application requires that developers must establish a method for incorporating essential information during application execution. Retrieval augmented generation (RAG) provides the capabilities you need for this.

RAG operates at a fundamental level through two main functions:

- Retrieval → finding relevant information from external data
- Generation → using an LLM to produce a coherent answer

The system goes beyond its memory base to search for the information it needs to answer questions.

The standard system architecture for an LLM with a vector database operates according to this sequence:

- *User input:* A user submits a query (e.g., a question or request).
- *Query embedding:* The input is converted into a vector representation using an embedding model.
- *Similarity search (retrieval step):* The system searches a vector database to find the most relevant chunks of data based on semantic similarity.
- *Context augmentation:* The system adds retrieved documents to the existing prompt for context purposes.
- *LLM generation:* The LLM processes the enriched prompt and generates a response grounded in the retrieved data.

This pipeline ensures that responses are not just fluent, but also contextually accurate and data backed.

The absence of RAG in LLMs can lead to the latter losing control over their factual output. RAG actual data

establishes response bases that enable continuous knowledge updates, because RAG assumes real-time data streaming from actual events and creates a better understanding through detailed output demonstration. It enables LLMs to operate as active knowledge systems because it switches their function from static knowledge production to active information generation.

## When to use this architecture

RAG functions as an essential tool for organisations that experience ongoing information transformation, require precise information and instruction verification, and have to search through extensive unstructured information sets. Organisations that want to standardise their operations through a common knowledge system must implement this structure because their requirements should match the module's capabilities for data retrieval and data discovery.

The architecture achieves maximum efficiency through its core function, which enables different parts of the system to work independently. The vector database acts as memory. The LLM acts as reasoning and generation. The orchestration layer (like LangChain) connects them seamlessly.

The system functions as a complete unit, which enables users to expand their operations while discovering and fixing issues to enhance their system performance.

The following sections will show you the steps needed to create this pipeline.

## Setup and tooling

Tooling must be established correctly before you start building your LLM-powered application. The testing process becomes easier when you have a clean setup as against a messy one, which creates unreliable testing environments.

LangChain operates as the main component of the system because it serves as the chief control system

that connects different parts of the infrastructure. The system establishes structured connections between LLMs and databases, which allows developers to construct their data processing systems through pipeline development instead of creating temporary solutions.

## Choosing an LLM provider

The first decision is selecting an LLM provider. Your main focus will determine which option you choose among these three possibilities:

- Ease of use → hosted APIs (e.g., OpenAI)
- Customisation → open source models (via Hugging Face)
- Cost and control → self-hosted models

The three available options provide different results based on their latency performance and cost structure. Most beginners should choose hosted API due to its simple implementation process.

## Selecting a vector database

You need to choose a vector database that can efficiently handle both storing and retrieving embeddings data. The following options serve as popular choices:

- FAISS – great for local development and fast similarity search
  - Pinecone – managed service with built-in scaling
  - Weaviate – flexible, supports metadata filtering and hybrid search
- Your choice will depend on scale:
- Small/local projects → FAISS
  - Production apps → Pinecone or Weaviate environment setup

A standard Python environment installation process follows this sequence of actions.

Install core libraries like LangChain LLM SDK (e.g., OpenAI or Hugging Face) and vector DB client. The system needs API keys to operate but you must protect them by using environment variables. Create a virtual environment that helps to keep your project libraries